1)

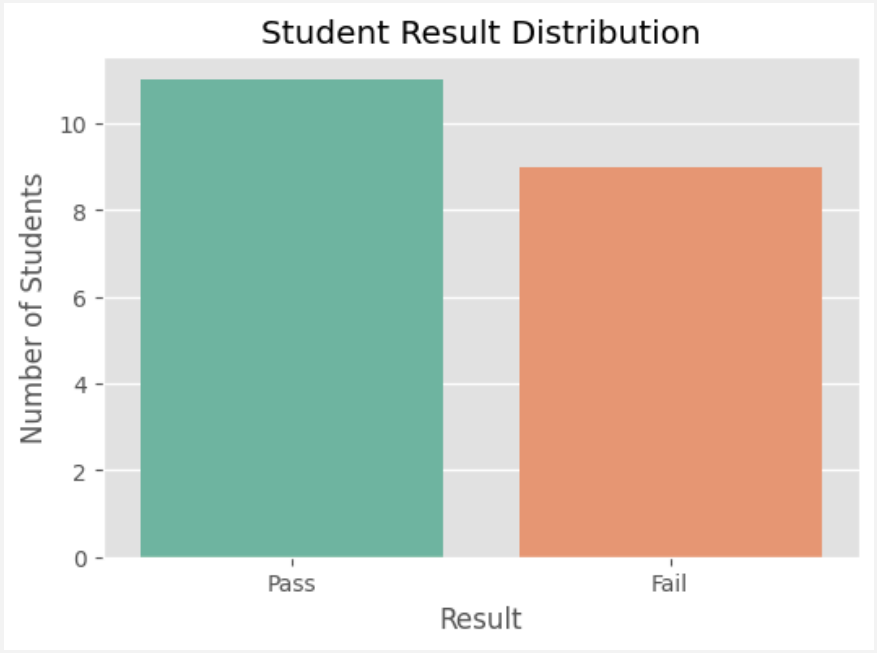
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

plt.style.use('ggplot')

import pandas as pd
df = pd.read_excel("Education_Analytics_Dataset.xlsx")

df
df.head()
df.tail()
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
df.columns
df.dtypes
df.info()
df.isnull().sum()
df.duplicated().sum()
df.describe()
df.mean(numeric_only=True)
df.median(numeric_only=True)
df.min(numeric_only=True)
df.max(numeric_only=True)
df.std(numeric_only=True)
df["Result"].value_counts()
df.corr(numeric_only=True)
```

```
Rows: 20
Columns: 5
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Student_ID      20 non-null     object
 1   Attendance      20 non-null     int64
 2   Study_Hours     20 non-null     int64
 3   Internal_Marks  20 non-null     int64
 4   Result          20 non-null     object
dtypes: int64(3), object(2)
memory usage: 932.0+ bytes
```

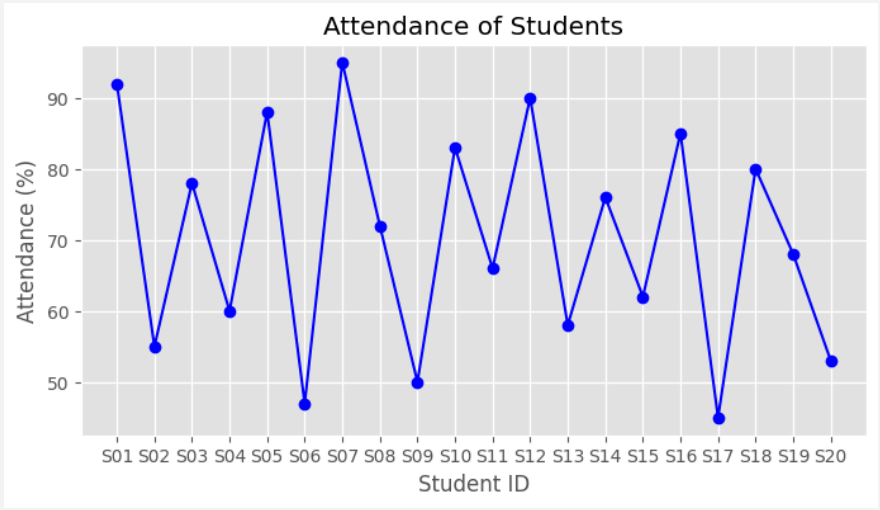
1 to 3 of 3 entries Filter

index	Attendance	Study_Hours	Internal_Marks
Attendance	1.0	0.975887865499992	0.997569410685541 2
Study_Hours	0.975887865499992	1.0	0.977298840508280 4
Internal_Marks	0.997569410685541 2	0.977298840508280 4	1.0

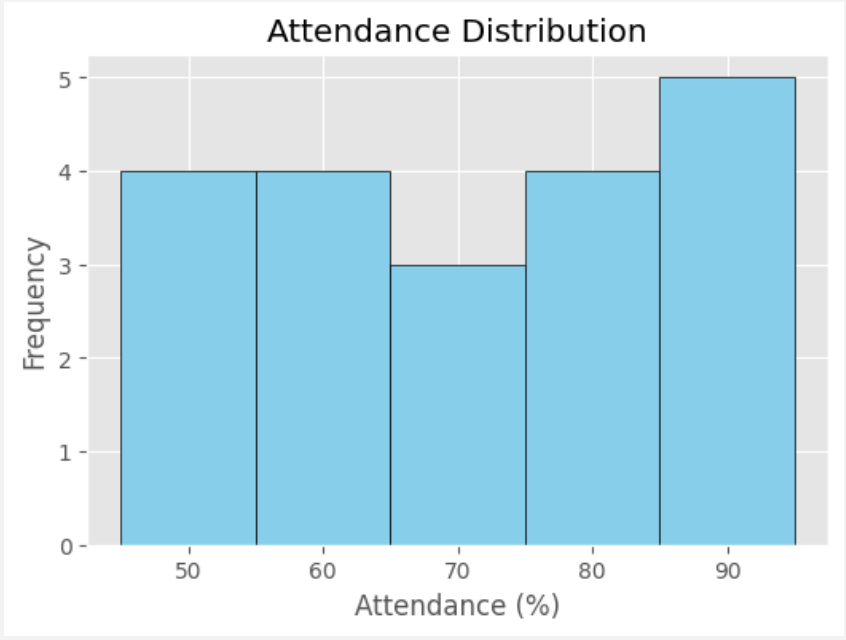
```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6,4))
sns.countplot(x="Result", data=df, palette="Set2")

plt.title("Student Result Distribution")
plt.xlabel("Result")
plt.ylabel("Number of Students")

plt.show()
```



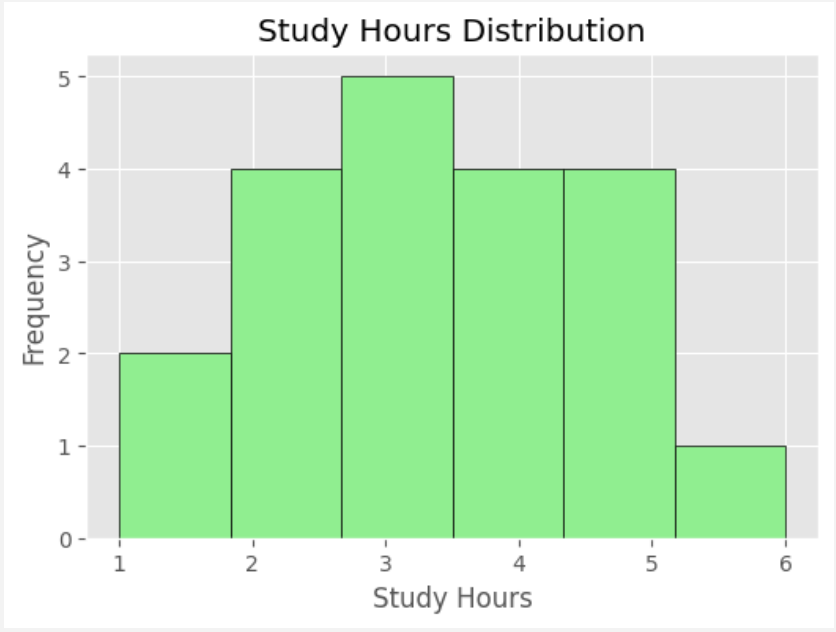
```
plt.figure(figsize=(8,4))

plt.plot(df["Student_ID"], df["Attendance"],
         marker='o', color='blue')

plt.title("Attendance of Students")
plt.xlabel("Student ID")
plt.ylabel("Attendance (%)")

plt.grid(True)
plt.show()
```



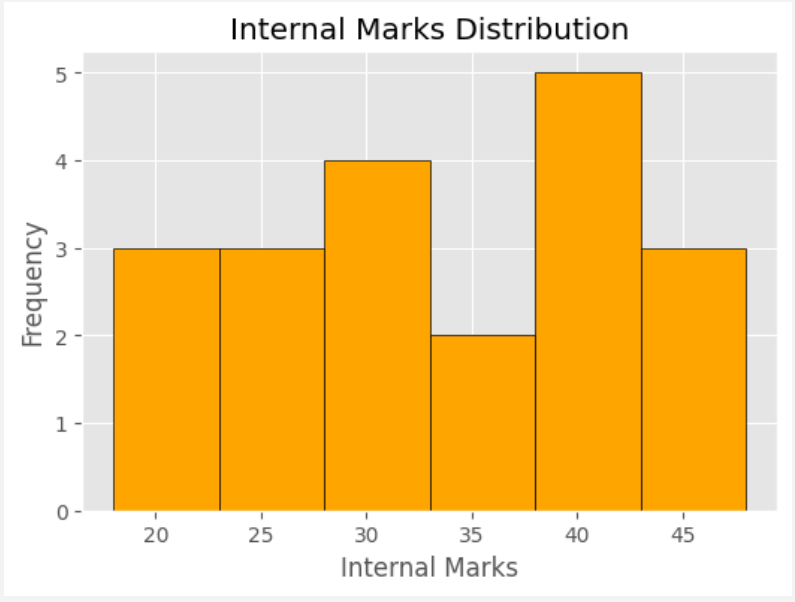

```
plt.figure(figsize=(6,4))

plt.hist(df["Attendance"],
         bins=5,
         color="skyblue",
         edgecolor="black")

plt.title("Attendance Distribution")
plt.xlabel("Attendance (%)")
plt.ylabel("Frequency")

plt.show()
```



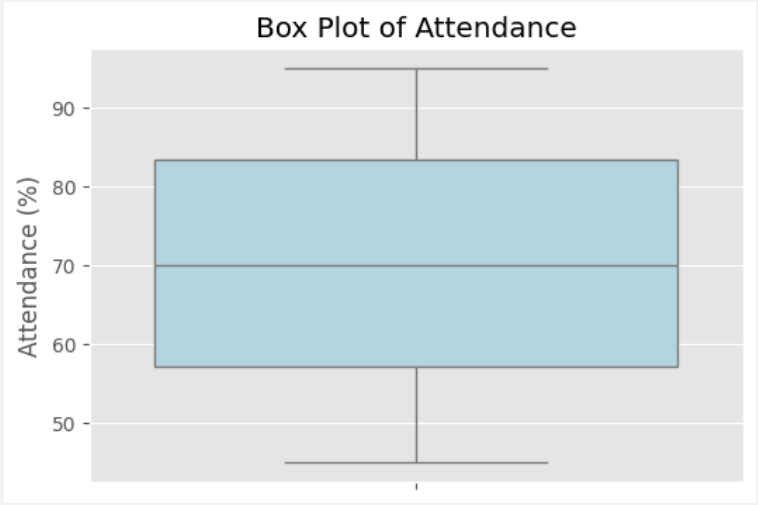

```
plt.figure(figsize=(6,4))

plt.hist(df["Study_Hours"],
         bins=6,
         color="lightgreen",
         edgecolor="black")

plt.title("Study Hours Distribution")
plt.xlabel("Study Hours")
plt.ylabel("Frequency")

plt.show()
```




```
plt.figure(figsize=(6,4))

plt.hist(df["Internal_Marks"],
         bins=6,
         color="orange",
         edgecolor="black")

plt.title("Internal Marks Distribution")
plt.xlabel("Internal Marks")
plt.ylabel("Frequency")

plt.show()
```

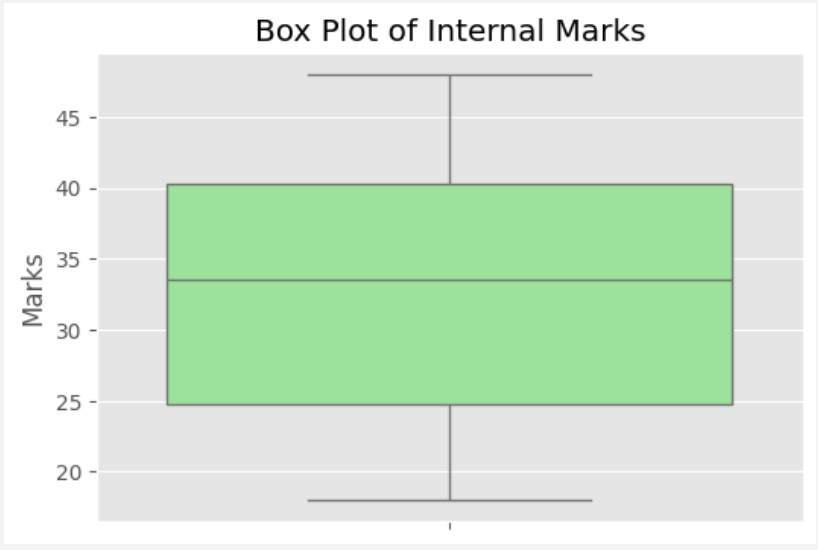



```
plt.figure(figsize=(6,4))

sns.boxplot(y=df["Attendance"], color="lightblue")

plt.title("Box Plot of Attendance")
plt.ylabel("Attendance (%)")

plt.show()
```

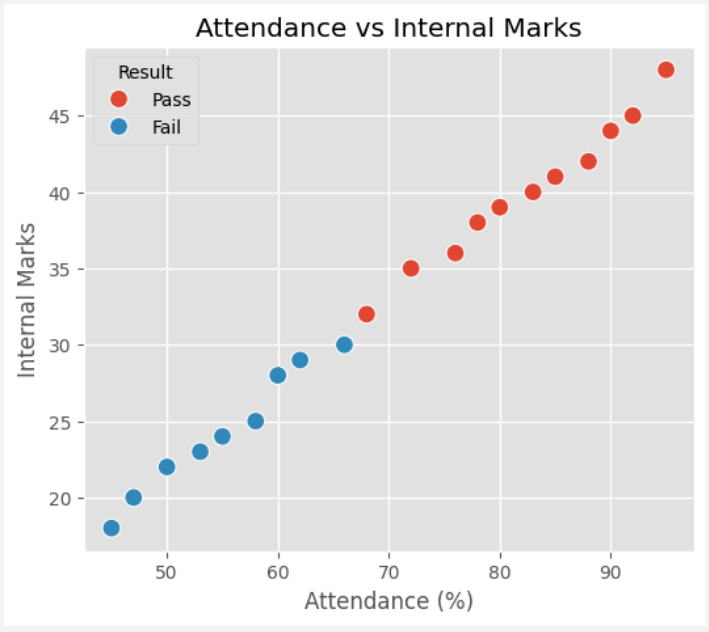



```
plt.figure(figsize=(6,4))

sns.boxplot(y=df["Internal_Marks"], color="lightgreen")
```

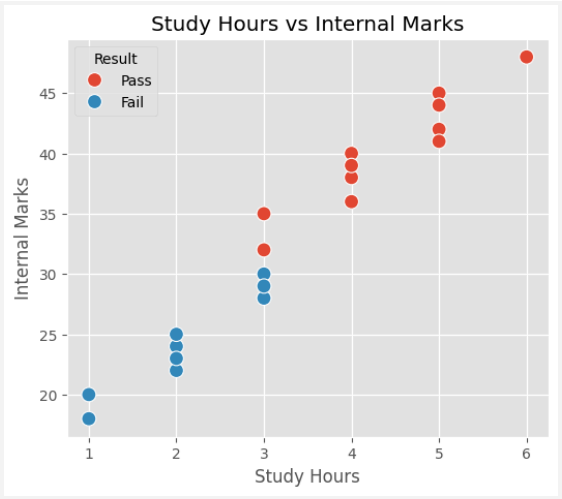
```
plt.title("Box Plot of Internal Marks")
plt.ylabel("Marks")

plt.show()
```



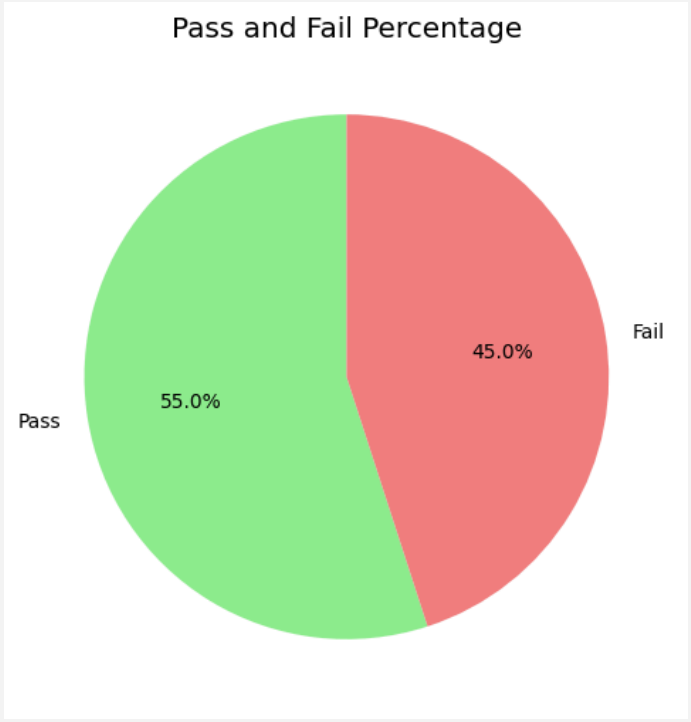

```
plt.figure(figsize=(6,5))

sns.scatterplot(
    x="Attendance",
    y="Internal_Marks",
    hue="Result",
    data=df,
    s=100
)

plt.title("Attendance vs Internal Marks")
plt.xlabel("Attendance (%)")
plt.ylabel("Internal Marks")

plt.show()
```




```
plt.figure(figsize=(6,5))

sns.scatterplot(
    x="Study_Hours",
    y="Internal_Marks",
    hue="Result",
    data=df,
    s=100
)
plt.title("Study Hours vs Internal Marks")
plt.xlabel("Study Hours")
plt.ylabel("Internal Marks")
plt.show()
```

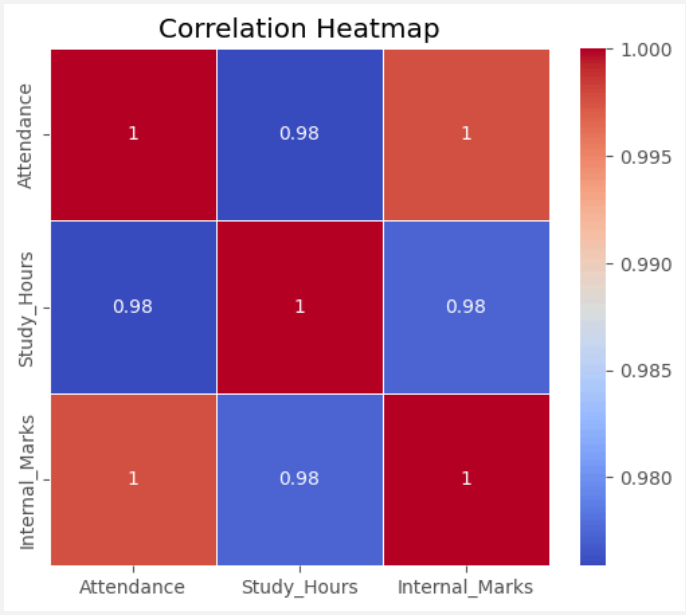


```
plt.figure(figsize=(6,6))

df["Result"].value_counts().plot(
    kind="pie",
    autopct="%1.1f%%",
    startangle=90,
    colors=["lightgreen","lightcoral"]
)

plt.title("Pass and Fail Percentage")
plt.ylabel("")

plt.show()
```



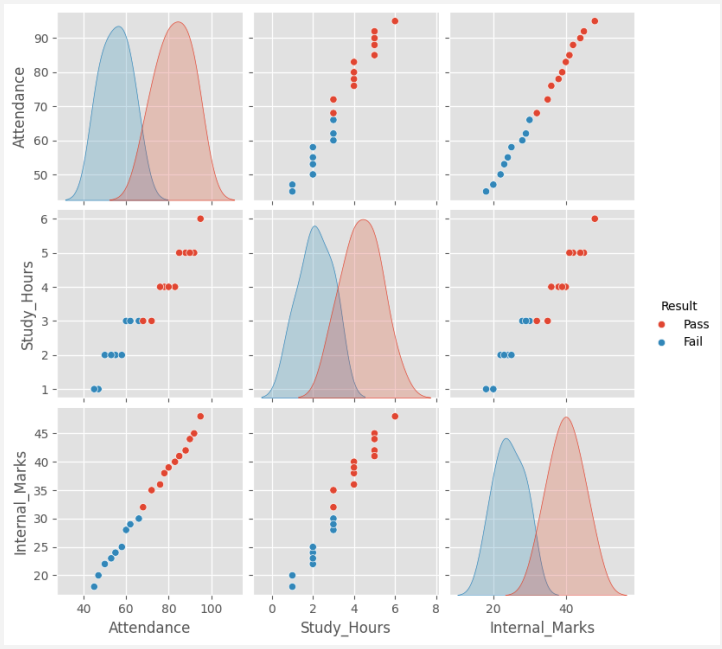

```
plt.figure(figsize=(6,5))

sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    cmap="coolwarm",
    linewidths=0.5
)

plt.title("Correlation Heatmap")
```

```
plt.show()
```



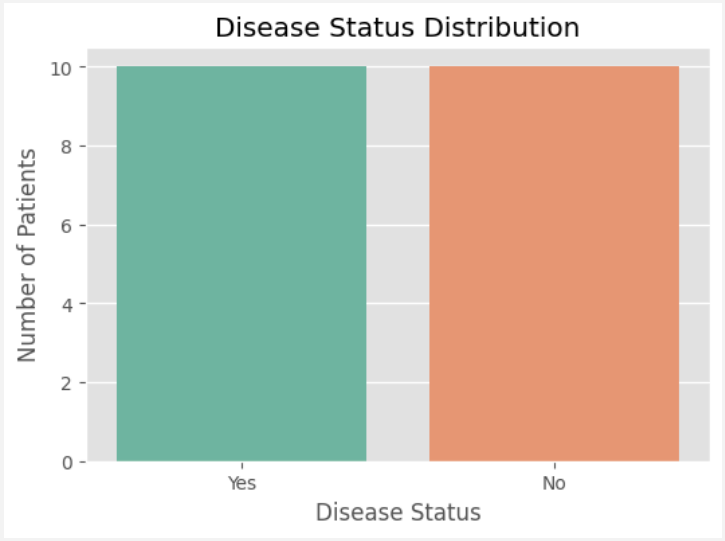

```
sns.pairplot(df, hue="Result")
```

```
plt.show()
```

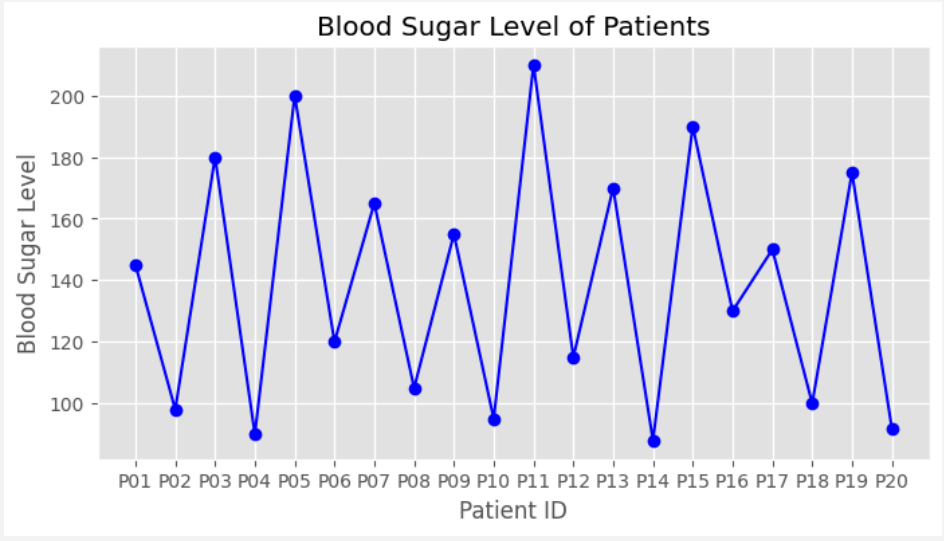


2)

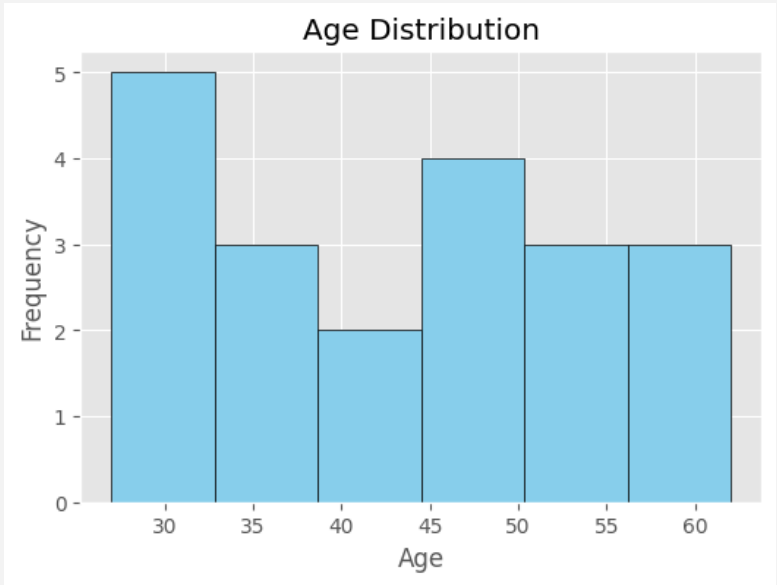
```
import pandas as pd

df = pd.read_excel("Healthcare_Analytics_Dataset.xlsx")


import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6,4))
sns.countplot(x="Disease_Status", data=df, palette="Set2")

plt.title("Disease Status Distribution")
plt.xlabel("Disease Status")
plt.ylabel("Number of Patients")

plt.show()
```



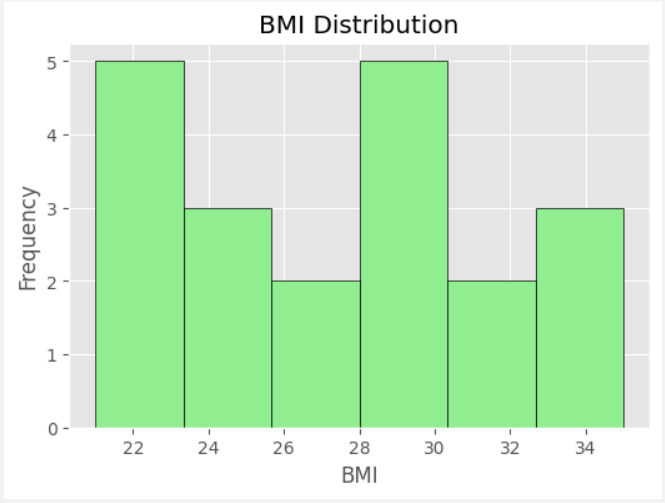

```
plt.figure(figsize=(8,4))

plt.plot(df["Patient_ID"], df["Sugar_Level"],
         marker='o', color='blue')

plt.title("Blood Sugar Level of Patients")
plt.xlabel("Patient ID")
plt.ylabel("Blood Sugar Level")

plt.grid(True)
plt.show()
```



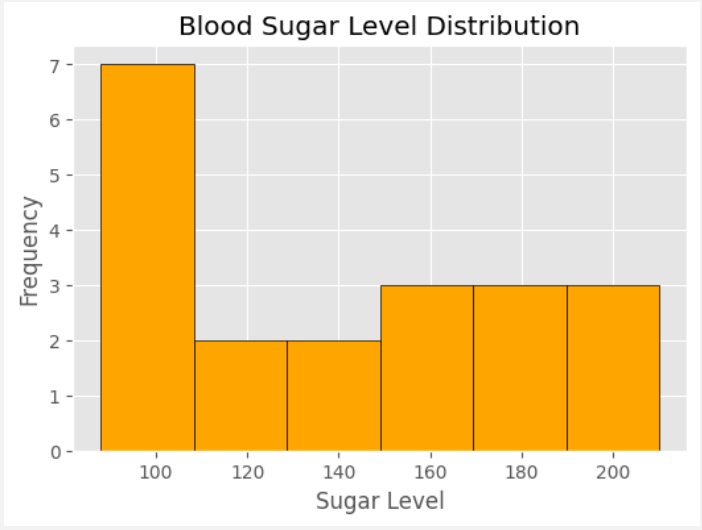

```
plt.figure(figsize=(6,4))

plt.hist(df["Age"],
         bins=6,
         color="skyblue",
         edgecolor="black")

plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")

plt.show()
```



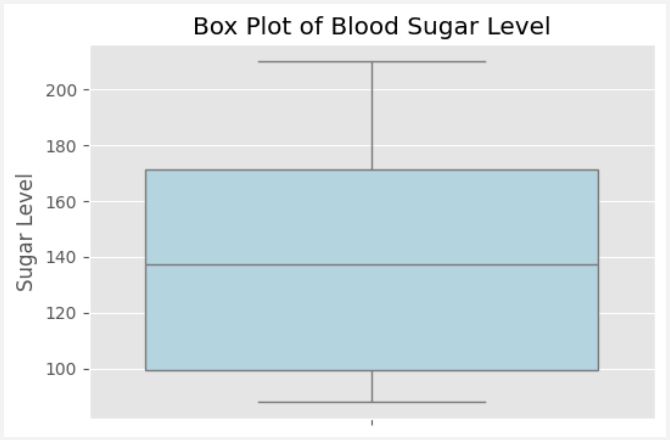
```
plt.figure(figsize=(6,4))

plt.hist(df["BMI"],
         bins=6,
         color="lightgreen",
         edgecolor="black")

plt.title("BMI Distribution")
plt.xlabel("BMI")
plt.ylabel("Frequency")

plt.show()
```




```
plt.figure(figsize=(6,4))

plt.hist(df["Sugar_Level"],
         bins=6,
         color="orange",
         edgecolor="black")

plt.title("Blood Sugar Level Distribution")
plt.xlabel("Sugar Level")
plt.ylabel("Frequency")

plt.show()
```

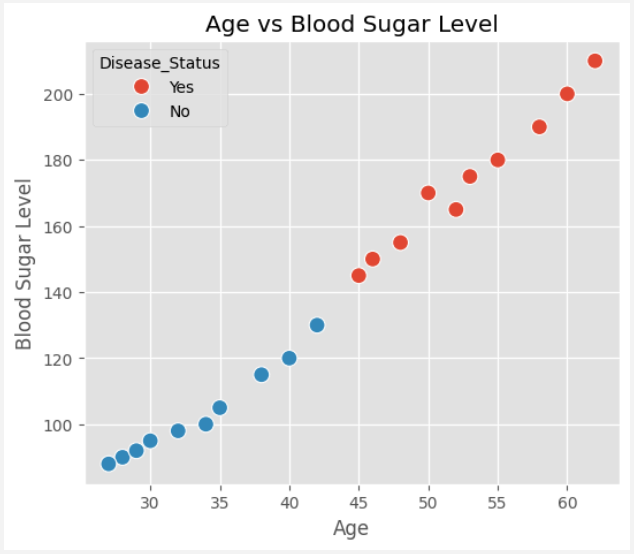



```
plt.figure(figsize=(6,4))

sns.boxplot(y=df["Sugar_Level"], color="lightblue")

plt.title("Box Plot of Blood Sugar Level")
plt.ylabel("Sugar Level")

plt.show()
```

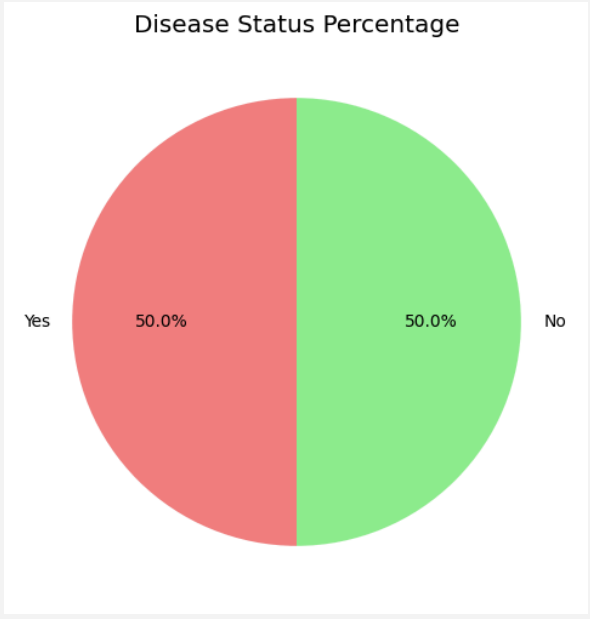



```
plt.figure(figsize=(6,5))

sns.scatterplot(
    x="Age",
    y="Sugar_Level",
    hue="Disease_Status",
    data=df,
    s=100
```

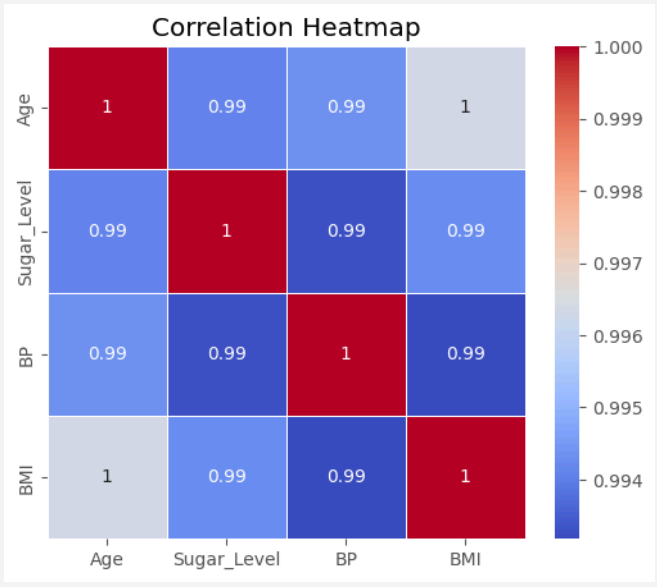
```
)

plt.title("Age vs Blood Sugar Level")
plt.xlabel("Age")
plt.ylabel("Blood Sugar Level")

plt.show()
```



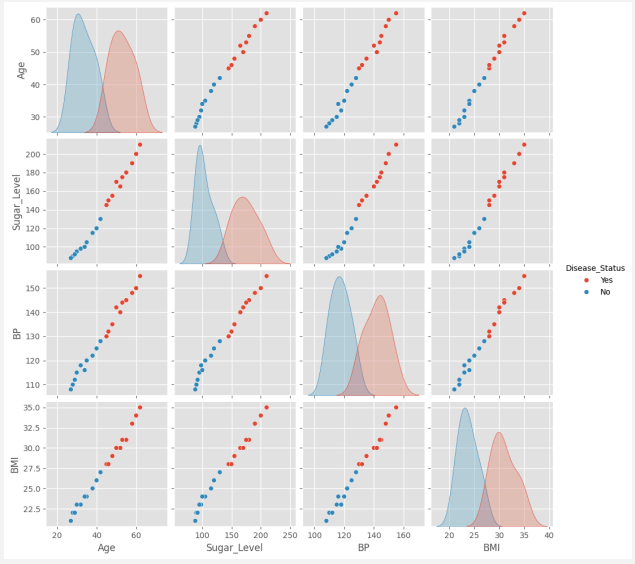

```
plt.figure(figsize=(6,6))

df["Disease_Status"].value_counts().plot(
    kind="pie",
    autopct="%1.1f%%",
    startangle=90,
    colors=["lightcoral","lightgreen"]
)

plt.title("Disease Status Percentage")
plt.ylabel("")

plt.show()
```




```
plt.figure(figsize=(6,5))

sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    cmap="coolwarm",
    linewidths=0.5
)

plt.title("Correlation Heatmap")

plt.show()
```




```
sns.pairplot(df, hue="Disease_Status")

plt.show()
```



3)

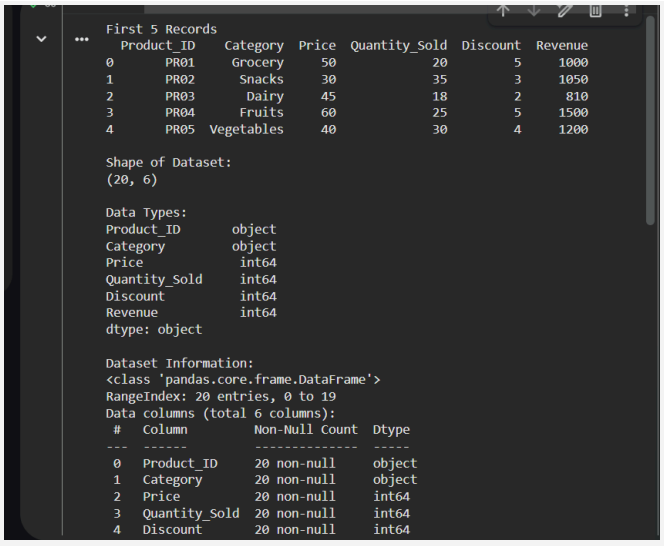
```
# ============================
# Import Libraries
# ============================
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ==========================
# Load Dataset
# ==========================
df = pd.read_excel("Retail_Sales_Dataset_Q3.xlsx")

# ==========================
# Display Dataset
# ==========================
print("First 5 Records")
print(df.head())

# ==========================
# Dataset Information
# ==========================
print("\nShape of Dataset:")
print(df.shape)

print("\nData Types:")
print(df.dtypes)

print("\nDataset Information:")
print(df.info())

# ==========================
# Missing Values
# ==========================
print("\nMissing Values")
print(df.isnull().sum())

# ==========================
# Duplicate Values
# ==========================
print("\nDuplicate Records")
print(df.duplicated().sum())

# Remove duplicates (if any)
df = df.drop_duplicates()

# ==========================
# Descriptive Statistics
```

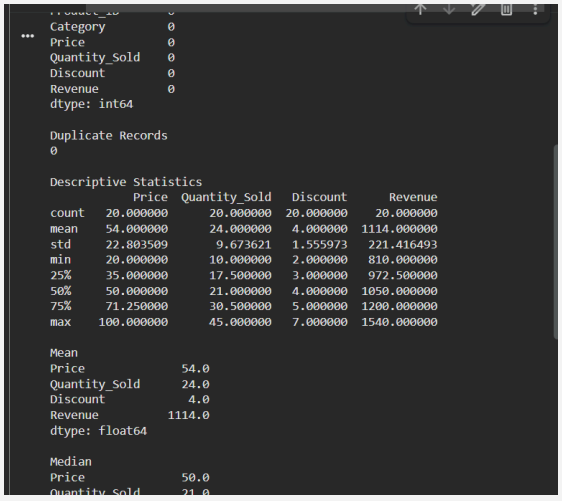
```
# ==========================
print("\nDescriptive Statistics")
print(df.describe())

# ==========================
# Mean, Median, Min, Max, Std
# ==========================
print("\nMean")
print(df.mean(numeric_only=True))

print("\nMedian")
print(df.median(numeric_only=True))

print("\nMinimum")
print(df.min(numeric_only=True))

print("\nMaximum")
print(df.max(numeric_only=True))

print("\nStandard Deviation")
print(df.std(numeric_only=True))

# ==========================
# Category-wise Revenue
# ==========================
print("\nRevenue by Category")
print(df.groupby("Category")["Revenue"].sum())

# ==========================
# Category-wise Quantity Sold
# ==========================
print("\nQuantity Sold by Category")
print(df.groupby("Category")["Quantity_Sold"].sum())

# ==========================
# Correlation Matrix
# ==========================
print("\nCorrelation Matrix")
print(df[["Price","Quantity_Sold","Discount","Revenue"]].corr())

# ==========================
# BAR CHART
# ==========================
plt.figure(figsize=(6,4))
```

```
df.groupby("Category")["Revenue"].sum().plot(kind="bar", color="skyblue")
plt.title("Revenue by Category")
plt.xlabel("Category")
plt.ylabel("Revenue")
plt.xticks(rotation=45)
plt.show()

# ==========================
# HISTOGRAM
# ==========================
plt.figure(figsize=(6,4))
plt.hist(df["Price"], bins=6, color="orange", edgecolor="black")
plt.title("Price Distribution")
plt.xlabel("Price")
plt.ylabel("Frequency")
plt.show()

# ==========================
# SCATTER PLOT
# ==========================
plt.figure(figsize=(6,4))
plt.scatter(df["Price"], df["Revenue"], color="green")
plt.title("Price vs Revenue")
plt.xlabel("Price")
plt.ylabel("Revenue")
plt.show()

# ==========================
# BOX PLOT
# ==========================
plt.figure(figsize=(6,4))
sns.boxplot(y=df["Revenue"])
plt.title("Revenue Box Plot")
plt.show()

# ==========================
# PIE CHART
# ==========================
plt.figure(figsize=(6,6))
df["Category"].value_counts().plot(
    kind="pie",
    autopct="%1.1f%%",
    startangle=90
)
```

```
plt.title("Category Distribution")
plt.ylabel("")
plt.show()

# ==========================
# HEATMAP
# ==========================
plt.figure(figsize=(6,4))
sns.heatmap(
    df[["Price","Quantity_Sold","Discount","Revenue"]].corr(),
    annot=True,
    cmap="Blues"
)
plt.title("Correlation Heatmap")
plt.show()
```

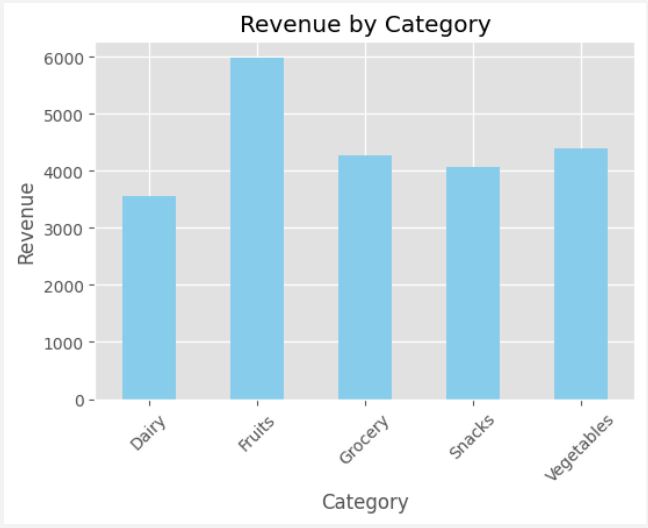


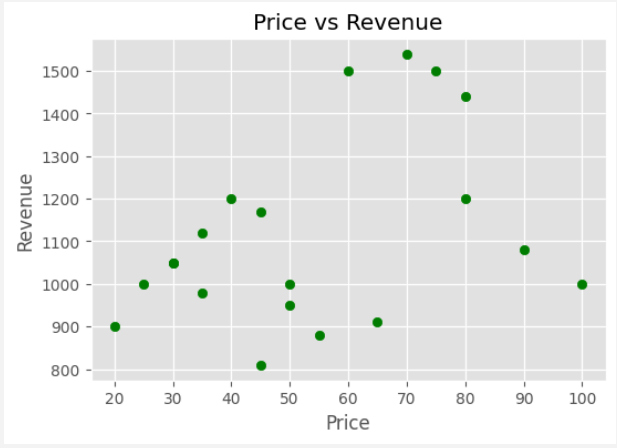
```
First 5 Records
  Product_ID    Category  Price  Quantity_Sold  Discount  Revenue
0       PR01     Grocery     50             20         5     1000
1       PR02      Snacks     30             35         3     1050
2       PR03       Dairy     45             18         2      810
3       PR04      Fruits     60             25         5     1500
4       PR05  Vegetables     40             30         4     1200

Shape of Dataset:
(20, 6)

Data Types:
Product_ID       object
Category         object
Price             int64
Quantity_Sold     int64
Discount          int64
Revenue           int64
dtype: object

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Product_ID     20 non-null     object
 1   Category       20 non-null     object
 2   Price          20 non-null     int64
 3   Quantity_Sold  20 non-null     int64
 4   Discount       20 non-null     int64
```

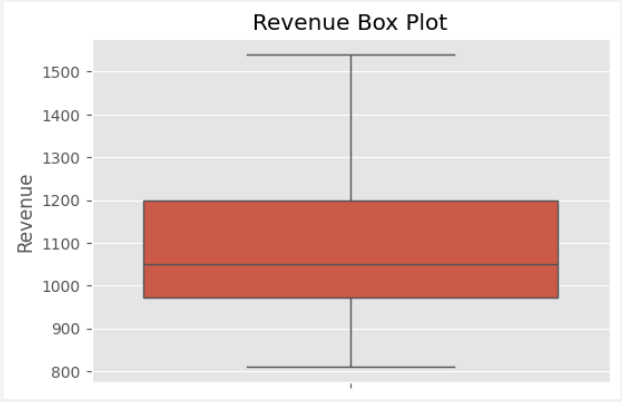


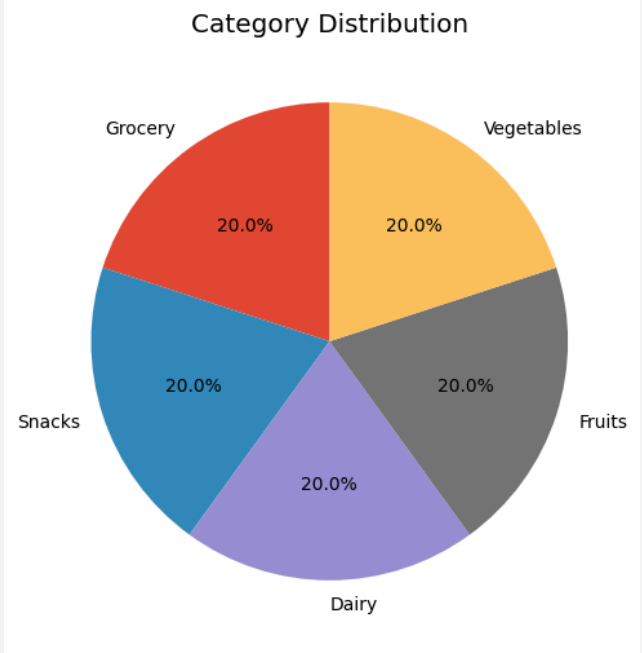
```
Category         0
Price            0
Quantity_Sold    0
Discount         0
Revenue          0
dtype: int64

Duplicate Records
0

Descriptive Statistics
            Price  Quantity_Sold   Discount      Revenue
count   20.000000      20.000000  20.000000    20.000000
mean    54.000000      24.000000   4.000000  1114.000000
std     22.803509       9.673621   1.555973   221.416493
min     20.000000      10.000000   2.000000   810.000000
25%     35.000000      17.500000   3.000000   972.500000
50%     50.000000      21.000000   4.000000  1050.000000
75%     71.250000      30.500000   5.000000  1200.000000
max    100.000000      45.000000   7.000000  1540.000000

Mean
Price             54.0
Quantity_Sold     24.0
Discount           4.0
Revenue         1114.0
dtype: float64

Median
Price             50.0
```

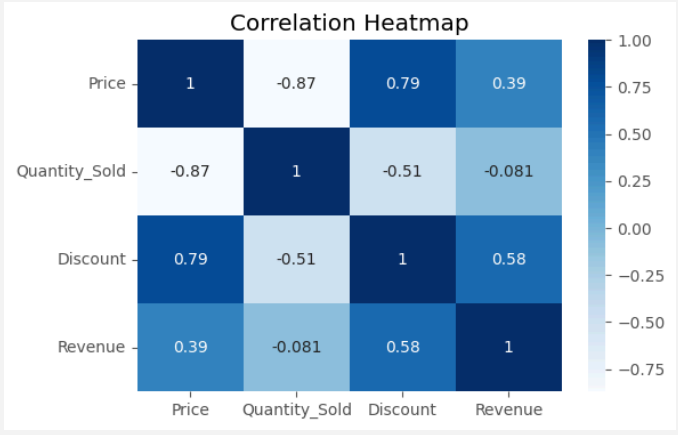
```
Minimum
Price             20
Quantity_Sold     10
Discount           2
Revenue          810
dtype: int64

Maximum
Price            100
Quantity_Sold     45
Discount           7
Revenue         1540
dtype: int64

Standard Deviation
Price            22.803509
Quantity_Sold     9.673621
Discount          1.555973
Revenue         221.416493
dtype: float64

Revenue by Category
Category
Dairy         3550
Fruits        5980
Grocery       4280
Snacks        4070
Vegetables    4400
Name: Revenue, dtype: int64

Quantity Sold by Category
```




Price vs Revenue
Revenue
Price
1500
1400
1300
1200
1100
1000
900
800
20
30
40
50
60
70
80
90
100


Revenue Box Plot
Revenue
1500
1400
1300
1200
1100
1000
900
800


Category Distribution
Grocery
Vegetables
20.0%
20.0%
20.0%
20.0%
Snacks
Fruits
20.0%
Dairy



4)

```
# ================================
# Banking Loan Approval Dataset EDA
# ================================

# Import Libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load Dataset
df = pd.read_excel("Banking_Loan_Approval_Dataset_Q4.xlsx")

# ------------------------------
# Display Dataset
# ------------------------------
print("First 5 Records:")
print(df.head())

# ------------------------------
# Shape of Dataset
# ------------------------------
print("\nNumber of Rows and Columns:")
print(df.shape)

# ------------------------------
# Data Types
# ------------------------------
print("\nData Types:")
print(df.dtypes)
```

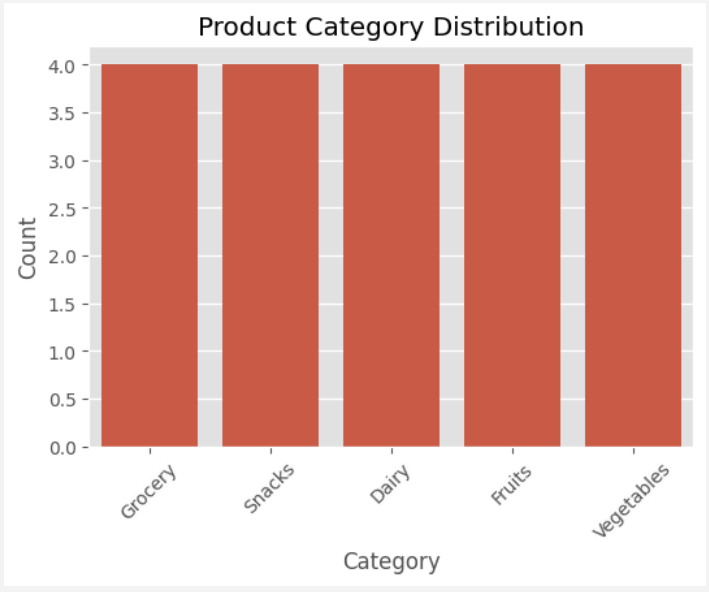
```
# ------------------------------
# Dataset Information
# ------------------------------
print("\nDataset Information:")
df.info()

# ------------------------------
# Missing Values
# ------------------------------
print("\nMissing Values:")
print(df.isnull().sum())

# ------------------------------
# Duplicate Values
# ------------------------------
print("\nDuplicate Records:")
print(df.duplicated().sum())

# Remove duplicates if any
df = df.drop_duplicates()

# ------------------------------
# Descriptive Statistics
# ------------------------------
print("\nDescriptive Statistics:")
print(df.describe())

# ------------------------------
# Mean, Median, Min, Max, Std
# ------------------------------
print("\nMean:")
print(df.mean(numeric_only=True))

print("\nMedian:")
print(df.median(numeric_only=True))

print("\nMinimum:")
print(df.min(numeric_only=True))

print("\nMaximum:")
print(df.max(numeric_only=True))

print("\nStandard Deviation:")
print(df.std(numeric_only=True))
```

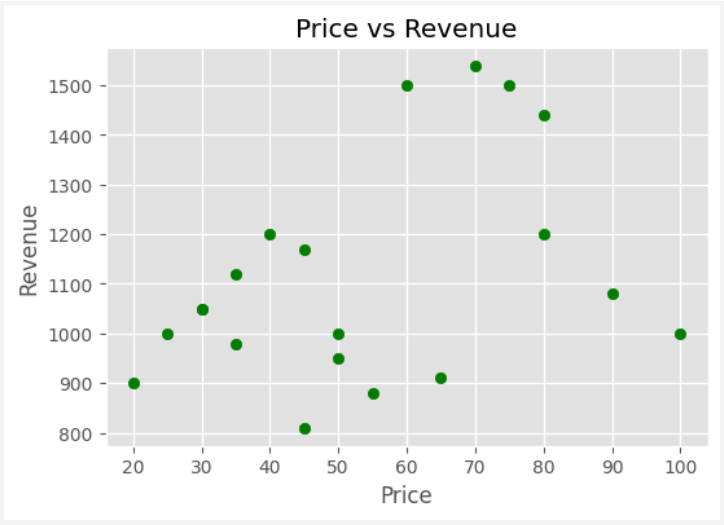
```
# ------------------------------
# Loan Status Count
# ------------------------------
print("\nLoan Status Count:")
print(df["Loan_Status"].value_counts())

# ------------------------------
# Employment Type Count
# ------------------------------
print("\nEmployment Type Count:")
print(df["Employment_Type"].value_counts())

# ------------------------------
# Average Income by Loan Status
# ------------------------------
print("\nAverage Income by Loan Status:")
print(df.groupby("Loan_Status")["Income"].mean())

# ------------------------------
# Average Credit Score by Loan Status
# ------------------------------
print("\nAverage Credit Score by Loan Status:")
print(df.groupby("Loan_Status")["Credit_Score"].mean())

# ------------------------------
# Correlation Matrix
# ------------------------------
print("\nCorrelation Matrix:")
print(df[["Income", "Credit_Score", "Loan_Amount"]].corr())

# ==============================
# Data Visualization
# ==============================

# 1. Bar Chart - Loan Status
plt.figure(figsize=(6,4))
sns.countplot(x="Loan_Status", data=df)
plt.title("Loan Approval Status")
plt.xlabel("Loan Status")
plt.ylabel("Count")
plt.show()

# 2. Histogram - Income
```

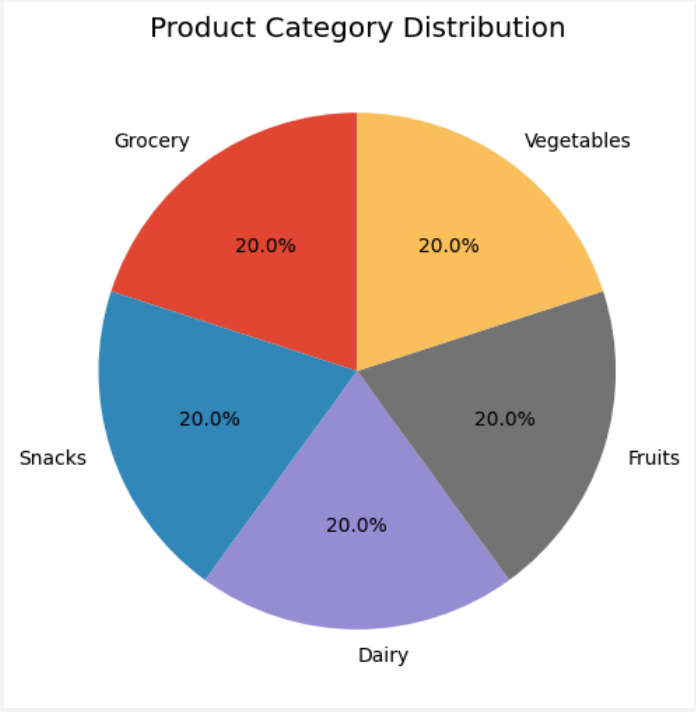
```
plt.figure(figsize=(6,4))
plt.hist(df["Income"], bins=6, edgecolor="black")
plt.title("Income Distribution")
plt.xlabel("Income")
plt.ylabel("Frequency")
plt.show()

# 3. Scatter Plot - Income vs Credit Score
plt.figure(figsize=(6,4))
plt.scatter(df["Income"], df["Credit_Score"], color="green")
plt.title("Income vs Credit Score")
plt.xlabel("Income")
plt.ylabel("Credit Score")
plt.show()

# 4. Box Plot - Credit Score
plt.figure(figsize=(6,4))
sns.boxplot(y=df["Credit_Score"])
plt.title("Box Plot of Credit Score")
plt.ylabel("Credit Score")
plt.show()

# 5. Pie Chart - Employment Type
plt.figure(figsize=(6,6))
df["Employment_Type"].value_counts().plot(
    kind="pie",
    autopct="%1.1f%%",
    startangle=90
)
plt.title("Employment Type Distribution")
plt.ylabel("")
plt.show()

# 6. Heatmap - Correlation
plt.figure(figsize=(6,4))
sns.heatmap(
    df[["Income", "Credit_Score", "Loan_Amount"]].corr(),
    annot=True,
    cmap="Blues"
)
plt.title("Correlation Heatmap")
plt.show()
```

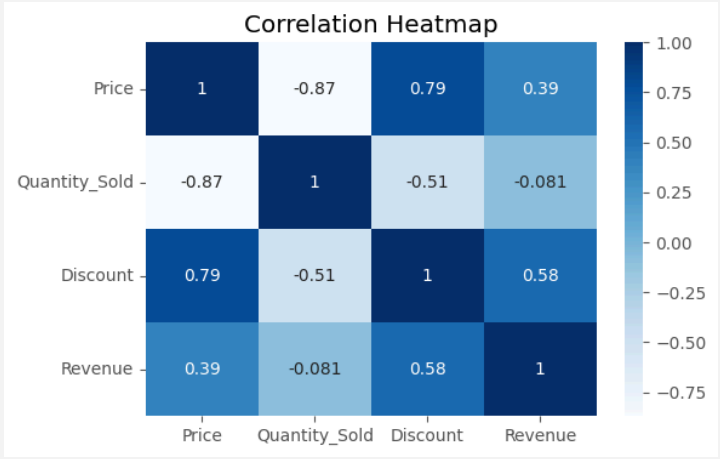
```
First 5 Records
  Product_ID    Category  Price  Quantity_Sold  Discount  Revenue
0       PR01     Grocery     50             20         5     1000
1       PR02      Snacks     30             35         3     1050
2       PR03       Dairy     45             18         2      810
3       PR04      Fruits     60             25         5     1500
4       PR05  Vegetables     40             30         4     1200

Shape of Dataset:
(20, 6)

Data Types:
Product_ID       object
Category         object
Price             int64
Quantity_Sold     int64
Discount          int64
Revenue           int64
dtype: object

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Product_ID     20 non-null     object
 1   Category       20 non-null     object
 2   Price          20 non-null     int64
 3   Quantity_Sold  20 non-null     int64
```

```
Discount           1.555973
Revenue          221.416493
dtype: float64

Category Distribution
Category
Grocery       4
Snacks        4
Dairy         4
Fruits        4
Vegetables    4
Name: count, dtype: int64

Average Revenue by Category
Category
Dairy          887.5
Fruits        1495.0
Grocery       1070.0
Snacks        1017.5
Vegetables    1100.0
Name: Revenue, dtype: float64

Correlation Matrix
                  Price  Quantity_Sold  Discount   Revenue
Price          1.000000      -0.868474  0.786174  0.388086
Quantity_Sold -0.868474       1.000000 -0.507018 -0.081335
Discount       0.786174      -0.507018  1.000000  0.575938
Revenue        0.388086      -0.081335  0.575938  1.000000
```


Product Category Distribution
Count
Category
Grocery
Snacks
Dairy
Fruits
Vegetables
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0


Price vs Revenue
Revenue
Price
800
900
1000
1100
1200
1300
1400
1500
20
30
40
50
60
70
80
90
100





5)

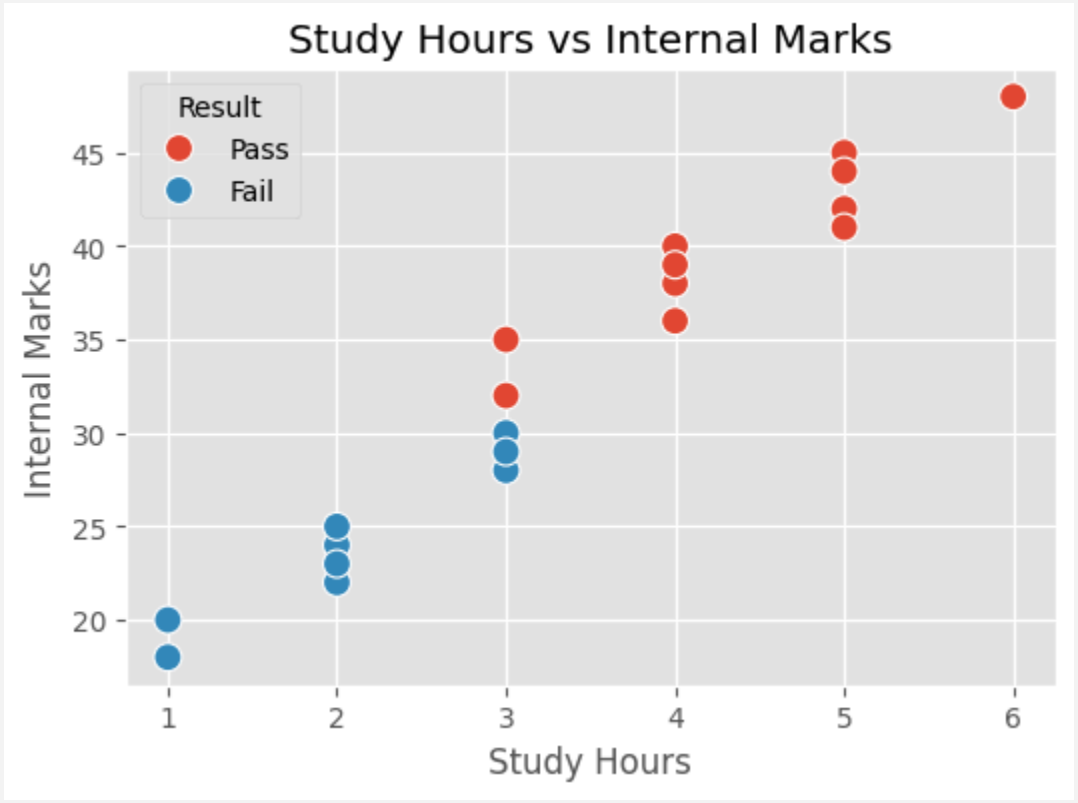
```
# ================================
# Agriculture Crop Yield Dataset EDA
# ================================

# Import Libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

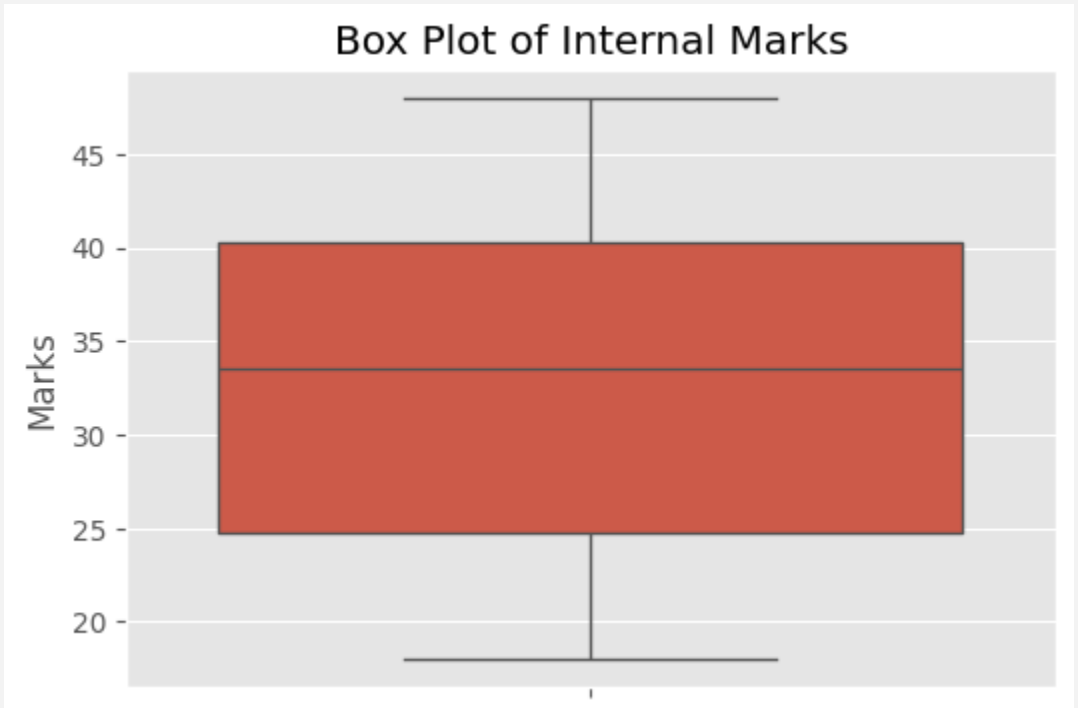
```
# Load Dataset
df = pd.read_excel("Agriculture_Crop_Yield_Dataset_Q5.xlsx")

# ------------------------------
# Display Dataset
# ------------------------------
print("First 5 Records:")
print(df.head())

# ------------------------------
# Shape of Dataset
# ------------------------------
print("\nNumber of Rows and Columns:")
print(df.shape)

# ------------------------------
# Data Types
# ------------------------------
print("\nData Types:")
print(df.dtypes)

# ------------------------------
# Dataset Information
# ------------------------------
print("\nDataset Information:")
df.info()

# ------------------------------
# Missing Values
# ------------------------------
print("\nMissing Values:")
print(df.isnull().sum())

# ------------------------------
# Duplicate Values
# ------------------------------
print("\nDuplicate Records:")
print(df.duplicated().sum())

# Remove duplicates if any
df = df.drop_duplicates()

# ------------------------------
```

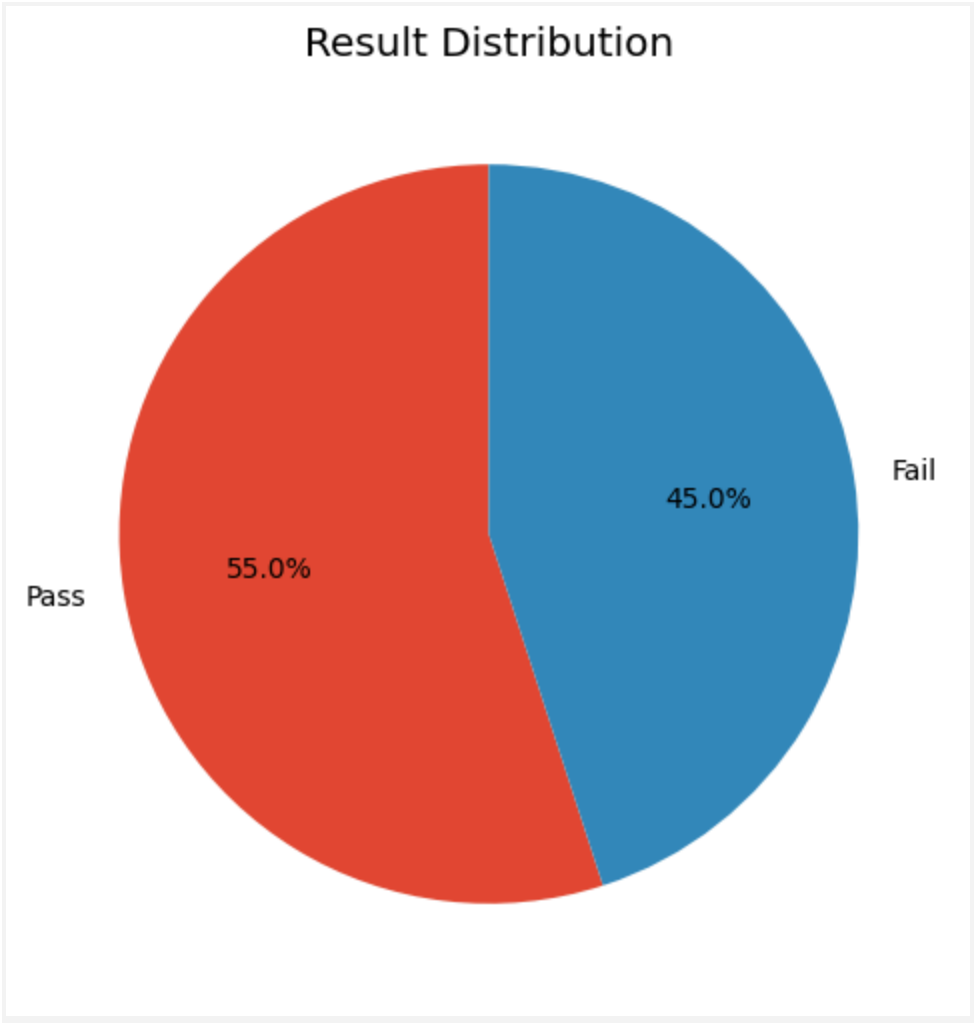
```
# Descriptive Statistics
# ------------------------------
print("\nDescriptive Statistics:")
print(df.describe())

# ------------------------------
# Mean, Median, Min, Max, Std
# ------------------------------
print("\nMean:")
print(df.mean(numeric_only=True))

print("\nMedian:")
print(df.median(numeric_only=True))

print("\nMinimum:")
print(df.min(numeric_only=True))

print("\nMaximum:")
print(df.max(numeric_only=True))

print("\nStandard Deviation:")
print(df.std(numeric_only=True))

# ------------------------------
# Crop Yield by Soil Type
# ------------------------------
print("\nAverage Crop Yield by Soil Type:")
print(df.groupby("Soil_Type")["Crop_Yield_kg"].mean())

# ------------------------------
# Correlation Matrix
# ------------------------------
print("\nCorrelation Matrix:")
print(df[["Rainfall_mm","Temperature","Fertilizer_kg","Crop_Yield_kg"]].corr())

# ==============================
# Data Visualization
# ==============================

# 1. Bar Chart - Average Crop Yield by Soil Type
plt.figure(figsize=(6,4))
df.groupby("Soil_Type")["Crop_Yield_kg"].mean().plot(kind="bar", color="green")
plt.title("Average Crop Yield by Soil Type")
plt.xlabel("Soil Type")
```

```
plt.ylabel("Crop Yield (kg)")
plt.show()

# 2. Histogram - Crop Yield
plt.figure(figsize=(6,4))
plt.hist(df["Crop_Yield_kg"], bins=6, edgecolor="black")
plt.title("Crop Yield Distribution")
plt.xlabel("Crop Yield (kg)")
plt.ylabel("Frequency")
plt.show()

# 3. Scatter Plot - Rainfall vs Crop Yield
plt.figure(figsize=(6,4))
plt.scatter(df["Rainfall_mm"], df["Crop_Yield_kg"], color="blue")
plt.title("Rainfall vs Crop Yield")
plt.xlabel("Rainfall (mm)")
plt.ylabel("Crop Yield (kg)")
plt.show()

# 4. Box Plot - Fertilizer Usage
plt.figure(figsize=(6,4))
sns.boxplot(y=df["Fertilizer_kg"])
plt.title("Box Plot of Fertilizer Usage")
plt.ylabel("Fertilizer (kg)")
plt.show()

# 5. Pie Chart - Soil Type Distribution
plt.figure(figsize=(6,6))
df["Soil_Type"].value_counts().plot(
    kind="pie",
    autopct="%1.1f%%",
    startangle=90
)
plt.title("Soil Type Distribution")
plt.ylabel("")
plt.show()

# 6. Heatmap - Correlation
plt.figure(figsize=(6,4))
sns.heatmap(
    df[["Rainfall_mm","Temperature","Fertilizer_kg","Crop_Yield_kg"]].corr(),
    annot=True,
    cmap="Greens"
)
```

```
plt.title("Correlation Heatmap")
plt.show()
```






Result Distribution
Fail
45.0%
55.0%
Pass